Informe Proyecto Paralela

Integrantes:

Miguel Angel Parada Rodríguez - 2201167

Sebastián Villamizar Saavedra - 2230078

David Steven Anaya González - 2211714

Sergio Andrés Pinzón Castellanos - 2202006

Nancy Liliana Saenz Moreno - 2224510

Introducción a la Programación Paralela

Universidad Industrial de Santander

2026-2

Nota: Tener en cuenta que este informe no es un entregable cómo tal sino una guía para trabajar posteriormente en el póster que se debe presentar el día 10 de noviembre de la otra semana. Además, es importante tener en cuenta las especificaciones de su equipo para el trabajo de cada modelo. Por poner un ejemplo, yo David Anaya , cuento con lo siguiente:

Todas las mediciones se hicieron en este computador:

Dato	Valor en su equipo
Modelo de procesador	AMD Ryzen 5 3450U with Radeon Vega Mobile Gfx (2.1 GHz base)
Núcleos físicos (Core(s) per socket × Socket(s))	4
Hilos por núcleo	2
Núcleos lógicos (nproc)	8
Valor de _OPENMP	202111

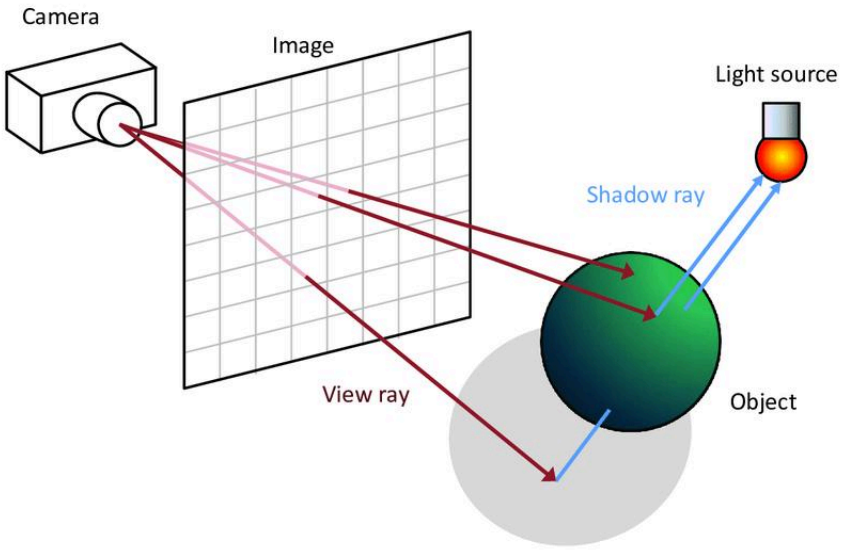
Compilador: gcc 16.2.0 (MSYS2 UCRT64), con las opciones -O2 -fopenmp. Sistema: Windows 11, 13.9 GB de RAM.

Fase 1: Raytracer serial

¿Qué es Ray Tracing?

Ray Tracing (trazado de rayos) es una técnica de renderizado que permite generar imágenes simulando el comportamiento de los rayos de luz en una escena tridimensional. En lugar de dibujar directamente los objetos mediante técnicas tradicionales de rasterización, Ray Tracing realiza un procedimiento basado en geometría y óptica.

La idea principal es simular el recorrido de la luz desde una cámara hacia los objetos de una escena para determinar qué color debería tener cada píxel de una imagen.



El objetivo es generar una imagen bidimensional que representa esa escena tridimensional. Para conseguirlo, el algoritmo realiza diferentes operaciones geométricas y físicas.

Por ejemplo, para determinar el color de un píxel, debe responder preguntas como:

1. ¿En qué dirección sale el rayo desde la cámara?
2. ¿El rayo intersecta algún objeto?
3. ¿Cuál es el objeto más cercano a la cámara?
4. ¿En qué punto de la superficie ocurrió la intersección?
5. ¿Qué orientación tiene la superficie?
6. ¿La fuente de luz ilumina directamente ese punto?
7. ¿Existe algún objeto que produzca una sombra?
8. ¿El material refleja parte de la luz?
9. ¿Qué color final corresponde a ese píxel?

Todas estas operaciones constituyen el trabajo computacional del Ray Tracer.

Ray Tracing es interesante para estudiar paralelismo porque la generación de una imagen puede dividirse en una gran cantidad de operaciones relativamente independientes.

Una imagen de:

- 640 × 480 tiene 307.200 píxeles.
- 1280 × 720 tiene 921.600 píxeles.
- 1920 × 1080 tiene 2.073.600 píxeles.
- 2560 × 1440 tiene 3.686.400 píxeles.

Para cada píxel se debe calcular un color.

Por lo tanto, podemos dividir el trabajo entre diferentes hilos del procesador. Por ejemplo, si tenemos una imagen de 1920 × 1080:

$$N_{píxeles} = 1920 \cdot 1080$$

$$N_{píxeles} = 2073600$$

Si tenemos ocho hilos lógicos disponibles, podemos distribuir el procesamiento de esos píxeles entre ellos. Sin embargo, hay una particularidad importante: no todos los píxeles requieren exactamente la misma cantidad de operaciones. Un píxel que representa una zona del cielo puede requerir únicamente calcular un color de fondo.

En cambio, un píxel que representa una esfera puede necesitar:

- Calcular intersecciones.
- Evaluar la iluminación.
- Lanzar rayos de sombra.
- Calcular reflexiones.
- Realizar intersecciones adicionales para los rayos reflejados.

Por eso, Ray Tracing permite estudiar no solamente la paralelización, sino también el problema del balanceo de carga. Para nuestro proyecto porque utilizaremos funcionalidades fundamentales:

- #pragma omp parallel
- #pragma omp for
- schedule(static)
- schedule(dynamic)
- schedule(guided)
- omp_get_wtime()
- omp_get_thread_num()

Antes de paralelizar, debemos construir un programa que genere imágenes correctamente utilizando un único flujo de ejecución. Dado que necesitamos una línea base (baseline) que nos permita comparar posteriormente los tiempos de ejecución. Si el programa serial tiene errores, los resultados de la versión paralela tampoco serán confiables.

¿Qué significa que el Raytracer sea serial?

Un programa serial ejecuta las operaciones siguiendo un único flujo de ejecución. La versión serial procesará un píxel después del otro. Es decir:

$$P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow \cdots \rightarrow P_{16}$$

Posteriormente, en la versión paralela, diferentes hilos podrán procesar diferentes píxeles simultáneamente.

¿Qué elementos debe tener nuestra escena?

La escena tendrá seis componentes fundamentales:

1. Vector tridimensional.
2. Rayo.
3. Esfera.
4. Plano.
5. Cámara.
6. Fuente de iluminación.

También agregaremos estructuras para materiales y objetos, porque necesitaremos representar distintos colores y propiedades reflectantes.

1. Vector tridimensional

Un vector tridimensional representa una cantidad con tres componentes:

$$\vec{v} = (x, y, z)$$

En nuestro RayTracer utilizaremos vectores para representar:

- Posiciones de los objetos.
- Direcciones de los rayos.
- Posición de la cámara.
- Posición de la luz.
- Normales de las superficies.
- Colores RGB.

La estructura en C será:

```
None
typedef struct {
    double x;
    double y;
    double z;
} Vec3;
```

Utilizaremos **double** para obtener una precisión numérica razonable en las operaciones geométricas.

Operaciones necesarias

Suma de vectores

$$\vec{a} + \vec{b} = (a_x + b_x, a_y + b_y, a_z + b_z)$$

Resta de vectores

$$\vec{a} - \vec{b} = (a_x - b_x, a_y - b_y, a_z - b_z)$$

Multiplicación por escalar

$$k\vec{a} = (ka_x, ka_y, ka_z)$$

Producto punto

$$\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y + a_z b_z$$

El producto punto será especialmente importante para calcular la iluminación.

Magnitud

$$\|\vec{v}\| = \sqrt{x^2 + y^2 + z^2}$$

Normalización

$$\hat{v} = \frac{\vec{v}}{\|\vec{v}\|}$$

Normalizar un vector significa convertirlo en un vector de longitud uno, conservando su dirección. Esto es importante porque las ecuaciones de iluminación y reflexión suelen trabajar con vectores unitarios.

2. **Rayo**

Un rayo es una representación matemática de una línea que tiene un origen y una dirección. Se define como:

$$R(t) = O + tD$$

Donde:

- O: origen del rayo.
- D: dirección del rayo.
- t: distancia paramétrica a lo largo del rayo.

Por ejemplo:

$$O = (0, 0, 0)$$

$$D = (0, 0, -1)$$

Entonces:

$$R(t) = (0, 0, 0) + t(0, 0, -1)$$

Para $t = 5$:

$$R(5) = (0, 0, -5)$$

Esto significa que el rayo ha avanzado cinco unidades en la dirección negativa del eje Z. En C:

```
None
typedef struct {
    Vec3 origin;
    Vec3 direction;
} Ray;
```

Esta estructura será utilizada para representar:

- Rayos primarios: salen de la cámara.
- Rayos de sombra: salen de una superficie hacia la luz.
- Rayos reflejados: continúan el recorrido de la luz reflejada.

3. Esfera

Una esfera es una superficie tridimensional definida por un centro y un radio.

Su ecuación es:

$$(x - C_x)^2 + (y - C_y)^2 + (z - C_z)^2 = r^2$$

Donde:

- C: centro de la esfera.
- r: radio.

Por ejemplo:

$$C = (0, 0, -5)$$

$$r = 1$$

Representa una esfera ubicada cinco unidades delante de la cámara, suponiendo que la cámara mira hacia el eje negativo Z.

En C:

```
None
typedef struct {
    Vec3 center;
    double radius;
```

```
    int material_id;
} Sphere;
```

El identificador del material nos permitirá asociar a cada esfera un color y una reflectividad.

Intersección rayo-esfera

Esta es una de las operaciones más importantes del Ray Tracer.

Necesitamos determinar si un rayo atraviesa una esfera.

Partimos de:

$$R(t) = O + tD$$

Sustituimos en la ecuación de la esfera:

$$\|O + tD - C\|^2 = r^2$$

Desarrollando:

$$(D \cdot D)t^2 + 2D \cdot (O - C)t + \|O - C\|^2 - r^2 = 0$$

Obtenemos una ecuación cuadrática:

$$at^2 + bt + c = 0$$

Con:

$$a = D \cdot D$$

$$b = 2D \cdot (O - C)$$

$$c = (O - C) \cdot (O - C) - r^2$$

El discriminante es:

$$\Delta = b^2 - 4ac$$

Tenemos tres posibilidades:

Discriminante	Interpretación
$\Delta < 0$	El rayo no intersecta la esfera
$\Delta = 0$	El rayo es tangente a la esfera
$\Delta > 0$	El rayo intersecta la esfera en dos puntos

Cuando existen dos intersecciones, calculamos:

$$t_1 = \frac{-b - \sqrt{\Delta}}{2a}$$

$$t_2 = \frac{-b + \sqrt{\Delta}}{2a}$$

Seleccionamos el menor valor positivo que esté dentro del intervalo permitido.

¿Por qué el menor?

Porque queremos encontrar el objeto visible más cercano a la cámara.

Si el rayo atraviesa una esfera, entra por un punto y sale por otro. La primera intersección corresponde a la superficie visible desde el origen del rayo.

Plano: el suelo de la escena

El suelo se representará mediante un plano infinito.

La ecuación general de un plano es:

$$n_x x + n_y y + n_z z + d = 0$$

Donde:

- $\vec{n}$: vector normal del plano.
- d: distancia algebraica respecto al origen.

Una forma más intuitiva es:

$$(P - P_0) \cdot N = 0$$

Donde:

- P: cualquier punto del plano.
- P_0: punto conocido del plano.
- N: normal.

Para nuestro suelo utilizaremos:

$$P_0 = (0, -1, 0)$$

$$N = (0, 1, 0)$$

Esto representa un suelo horizontal ubicado en:

$$y = -1$$

La normal apunta hacia arriba.

Intersección rayo-plano

Tenemos:

$$R(t) = O + tD$$

Sustituimos:

$$(O + tD - P_0) \cdot N = 0$$

Desarrollamos:

$$(O - P_0) \cdot N + t(D \cdot N) = 0$$

Despejamos:

$$t = \frac{(P_0 - O) \cdot N}{D \cdot N}$$

Si:

$$D \cdot N = 0$$

El rayo es paralelo al plano y no existe una intersección única. Si t es positivo, tenemos una posible intersección delante del origen del rayo. Esta operación permitirá que los rayos de la cámara detecten el suelo y que podamos calcular sombras sobre él.

4. Cámara

La cámara determina desde dónde observamos la escena y hacia dónde apuntamos. En nuestro caso, utilizaremos una cámara de perspectiva. La estructura será:

```
None
typedef struct {
    Vec3 position;
    Vec3 lower_left_corner;
    Vec3 horizontal;
    Vec3 vertical;
} Camera;
```

La cámara necesita:

- Posición.
- Campo de visión (Field of View, FOV).
- Relación de aspecto.
- Dirección de observación.

¿Qué es FOV?

El FOV determina cuánto espacio de la escena puede observar la cámara. Un FOV pequeño produce una imagen con un encuadre más cerrado. Pero un FOV grande permite observar una región más amplia.

Para nuestro experimento utilizaremos inicialmente:

$$FOV = 60^{\circ}$$

La relación de aspecto será:

$$A = \frac{W}{H}$$

Para una imagen de 1280 × 720**:**

$$A = \frac{1280}{720} = 1.7778$$

¿Cómo se genera un rayo por píxel?

Este procedimiento es esencial para entender cómo se paraleliza el programa. Supongamos que tenemos una imagen de ancho W y alto H.

Cada píxel tiene una posición:

$$(i, j)$$

Normalizamos sus coordenadas:

$$u = \frac{i+\alpha}{W-1}$$

$$v = \frac{j+\beta}{H-1}$$

Donde:

- i: coordenada horizontal.
- j: coordenada vertical.
- α, β: desplazamientos utilizados para antialiasing.

La dirección del rayo se calcula mediante:

$$D(u, v) = LLC + uH + vV - C$$

Donde:

- LLC: esquina inferior izquierda del plano de imagen.
- H: vector horizontal de la cámara.
- V: vector vertical.
- C: posición de la cámara.

Finalmente:

$$R(t) = C + tD(u, v)$$

Así obtenemos un rayo diferente para cada píxel**.**

Fuente de iluminación puntual

La iluminación puntual representa una fuente de luz situada en una posición específica del espacio.

La luz iluminará las superficies dependiendo de:

- Su posición.
- La distancia hasta el objeto.
- La orientación de la superficie.
- La presencia de obstáculos.

Estructura:

```
None
typedef struct {
    Vec3 position;
    Vec3 intensity;
```

```
} PointLight;
```

La intensidad tendrá tres componentes RGB.

Iluminación difusa: modelo Lambert

Ahora vamos a introducir el modelo de iluminación que permitirá que los objetos tengan una apariencia tridimensional.

El modelo Lambert representa la reflexión difusa de la luz.

La intensidad depende del ángulo entre:

- La normal de la superficie.
- La dirección hacia la fuente de luz.

La ecuación es:

$$I_d = k_d I_L \max(0, N \cdot L)$$

Donde:

- I_d: intensidad difusa.
- k_d: coeficiente difuso del material.
- I_L: intensidad de la luz.
- N: normal unitaria de la superficie.
- L: dirección unitaria hacia la luz.

¿Por qué utilizamos el producto punto?

El producto punto entre dos vectores unitarios es:

$$N \cdot L = \cos(\theta)$$

Donde θ es el ángulo entre ellos.

Por tanto:

Ángulo	Producto punto	Iluminación
0°	1	Máxima
30°	0.866	Alta
60°	0.5	Media
90°	0	Nula
Mayor a 90°	Negativo	Se considera cero

Esto tiene sentido físico: cuando la luz llega perpendicularmente a la superficie, la iluminación difusa es máxima. Cuando llega desde detrás de la superficie, no debe iluminar directamente. En nuestro programa utilizaremos:

```
None
double intensidad = fmax(0.0, dot(normal, direccion_luz));
```

Y multiplicamos este valor por el color de la superficie y la intensidad de la luz. También añadiremos una pequeña iluminación ambiental para evitar que las zonas no iluminadas queden completamente negras.

6. Sombras: rayos secundarios

Las sombras son fundamentales para que nuestra escena tenga profundidad visual. Para calcularlas, debemos determinar si existe algún objeto entre el punto iluminado y la fuente de luz. Supongamos que un rayo primario intersecta una esfera. Obtenemos el punto de intersección:

$$P = O + tD$$

Desde ese punto generamos un segundo rayo dirigido hacia la luz:

$$D_s = \frac{L - P}{\|L - P\|}$$

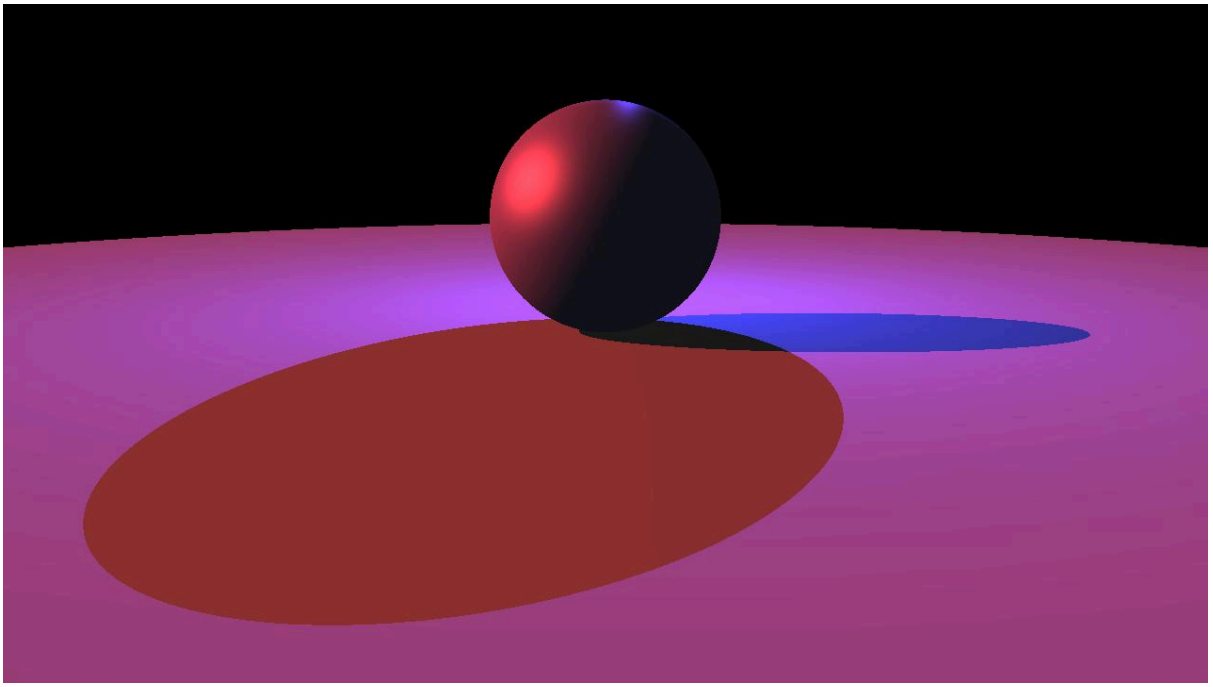
Este es el rayo de sombra. Su origen es ligeramente desplazado:

$$O_s = P + \epsilon N$$

Donde ϵ es un valor pequeño que evita que el rayo intersecta inmediatamente la misma superficie. Ahora comprobamos si el rayo de sombra intersecta algún otro objeto antes de llegar a la luz.

Si existe una intersección: $Hay\ Sombra$

Si no existe: $La\ superficie\ recibe\ luz\ directa$



Esto introduce una operación adicional por cada punto iluminado. Por eso, la cantidad de trabajo por píxel comienza a aumentar.

7. Reflexión recursiva

Este componente hará que el proyecto tenga suficiente carga computacional para comparar las diferentes estrategias de OpenMP. La reflexión permite simular superficies que reflejan parte de la iluminación de su entorno.

Por ejemplo:

- Una esfera metálica.
- Una superficie parcialmente reflectante.
- Una esfera que refleja el suelo y otras esferas.

La dirección del rayo reflejado se calcula mediante:

$$R = D - 2(D \cdot N)N$$

Donde:

- D: dirección incidente.
- N: normal de la superficie.
- R: dirección reflejada.

Para obtener el color reflejado, lanzamos un nuevo rayo desde el punto de intersección. Este proceso vuelve a ejecutar el algoritmo de intersección e iluminación. Por eso hablamos de recursividad. La función que calcula el color de un rayo puede llamar nuevamente a sí misma para calcular la contribución del rayo reflejado.

Podemos expresarlo así:

$$C(P) = (1 - k_r)C_{local} + k_r C_{reflejado}$$

Donde:

- $C(P)$: color final del punto.
- C_{local}: color obtenido mediante iluminación directa y ambiental.
- $C_{reflejado}$: color calculado mediante el rayo reflejado.
- k_r: coeficiente de reflectividad.

¿Por qué necesitamos limitar la recursividad?

Si cada rayo genera otro rayo reflejado indefinidamente, el programa podría continuar ejecutando operaciones sin terminar.

Por eso definimos una profundidad máxima:

$$D_{\max} = 3$$

Por ejemplo:

```
None
Rayo primario
  |
  +---- Rayo reflejado 1
        |
        +---- Rayo reflejado 2
              |
              +---- Rayo reflejado 3
                    |
                    +---- Termina
```

En el proyecto comenzaremos con una profundidad de tres rebotes. Posteriormente podremos probar cuatro o cinco para aumentar el costo computacional.

8. Antialiasing: varias muestras por píxel

Una imagen generada mediante Ray Tracing puede presentar bordes dentados. Esto sucede porque cada píxel representa una región de la imagen, pero si únicamente lanzamos un rayo por píxel, estamos realizando una única evaluación puntual. El antialiasing permite suavizar estos bordes mediante múltiples muestras.

En lugar de lanzar un rayo, lanzamos varios rayos ligeramente desplazados dentro de cada píxel. Por ejemplo:

- 1 muestra: sin antialiasing.

- 4 muestras: calidad básica.
- 16 muestras: mayor suavidad.
- 64 muestras: calidad superior.

El color final se obtiene mediante el promedio:

$$C_{\text{final}} = \frac{1}{S} \sum_{s=1}^{S} C_s$$

Donde:

- S: número de muestras.
- C_s: color calculado para cada muestra.

Si utilizamos 16 muestras por píxel**:**

$$N_{rayosprimarios} = W \cdot H \cdot 16$$

Para una imagen de 1280 × 720:

$$N_{rayosprimarios} = 1280 \cdot 720 \cdot 16$$

$$N_{rayosprimarios} = 14745600$$

Y esto sin contar los rayos secundarios de sombras y reflexión. Como puedes observar, la carga computacional aumenta considerablemente. Esta será una de las principales herramientas para conseguir que el renderizado tenga una duración medible.

Implementación

https://github.com/DSag31/Proyecto_Final_Programaci-n_Paralela/blob/main/RayTracer_Serial/raytracer_serial.c

¿Por qué utilizaremos PPM?

PPM significa Portable Pixmap.

Es un formato de imagen muy sencillo que permite almacenar imágenes en RGB.

Tiene varias ventajas para nuestro proyecto:

- No necesitamos bibliotecas externas para guardar imágenes.
- Podemos generar el archivo directamente desde C.
- Es fácil de inspeccionar.
- Podemos concentrarnos en el algoritmo de Ray Tracing.
- No necesitamos incorporar dependencias como OpenCV o stb_image.

Utilizaremos el formato binario P6.

Su estructura es:

```
None
1920 1080

255
```

Después de esa cabecera se almacenan los valores RGB de cada píxel, utilizando un byte por canal.

Explicación de las decisiones importantes del código

Hay varias decisiones que quiero que comprendas porque después serán importantes para el análisis PCAM.

Primera decisión: utilizar arreglos de esferas y materiales.

En lugar de escribir manualmente una función diferente para cada esfera, utilizamos:

```
None
Sphere spheres[NUM_SPHERES];
```

Esto permite recorrer todos los objetos con un ciclo.

Por ejemplo:

```
None
for (int i=0; i<NUM_SPHERES; i++) {

    // Comprobar interseccion

}
```

Si mañana queremos agregar diez esferas más, no necesitamos cambiar el algoritmo de intersección.

Únicamente agregamos nuevos elementos a la escena.

Segunda decisión: separar closest_hit() de ray_color().

La función closest_hit() determina qué objeto intersecta primero el rayo.

No calcula iluminación, sombras ni reflexión.

Esto es importante porque una misma operación geométrica se necesita en diferentes situaciones:

- Para los rayos primarios.
- Para los rayos de sombra.
- Para los rayos reflejados.

Así evitamos duplicar código.

Tercera decisión: cada píxel tiene su propio generador aleatorio.

Observa:

```
None
uint32_t state = hash32(

    (uint32_t)(x+1)*73856093u ^

    (uint32_t)(y+1)*19349663u
```

```
);
```

Esto permite obtener secuencias diferentes para distintos píxeles.

¿Por qué es importante?

Cuando utilicemos OpenMP, varios hilos ejecutarán simultáneamente render_pixel().

Si todos utilizaran una única variable aleatoria global, podrían modificarla simultáneamente, generando condiciones de carrera.

En cambio, cada llamada a render_pixel() mantiene su propia variable state.

Por tanto, no necesitamos proteger el generador aleatorio mediante un critical, un atomic o un lock.

Cuarta decisión: el framebuffer se almacena en un arreglo unidimensional.

Aunque conceptualmente la imagen tiene dos dimensiones, utilizaremos:

```
None
Vec3 *framebuffer;
```

Para acceder al píxel:

$$index = y \cdot WIDTH + x$$

Esto permite almacenar todos los píxeles consecutivamente en memoria.

También facilita mucho la paralelización porque podemos distribuir un rango de índices entre diferentes hilos.

Quinta decisión: el cálculo del color está separado del almacenamiento.

La función:

```
None
Vec3 render_pixel(...)
```

calcula el color, pero no escribe directamente en el archivo.

Devuelve el resultado para que el programa principal lo almacene en el framebuffer.

Esta separación es deliberada.

Más adelante podremos hacer que cada hilo calcule píxeles diferentes y escriba en posiciones diferentes del arreglo, sin necesidad de sincronizar cada escritura.

Qué se midió

Se usó el raytracer serial (raytracer_serial.c), es decir, la versión que trabaja con un solo hilo. Para que las mediciones se puedan comparar, siempre se dibujó la misma escena (semilla **2073120322**): una imagen de 1280 × 720 píxeles, con 32 muestras por píxel, hasta 3 reflejos y 6 esferas.

El programa se ejecutó 5 veces y se tomó la **mediana** (el valor del medio al ordenarlos), porque no se deja afectar por una corrida que salga anormalmente lenta o rápida. El tiempo mide solo el cálculo de la imagen, no el guardado del archivo.

Corrida	Tiempo (s)
1	6.982
2	7.042
3	7.071
4	7.008
5	6.979
Mediana	**7.008**

Las 5 corridas variaron menos de un 2 % entre sí (de 6.979 a 7.071 s), así que la medida es estable.

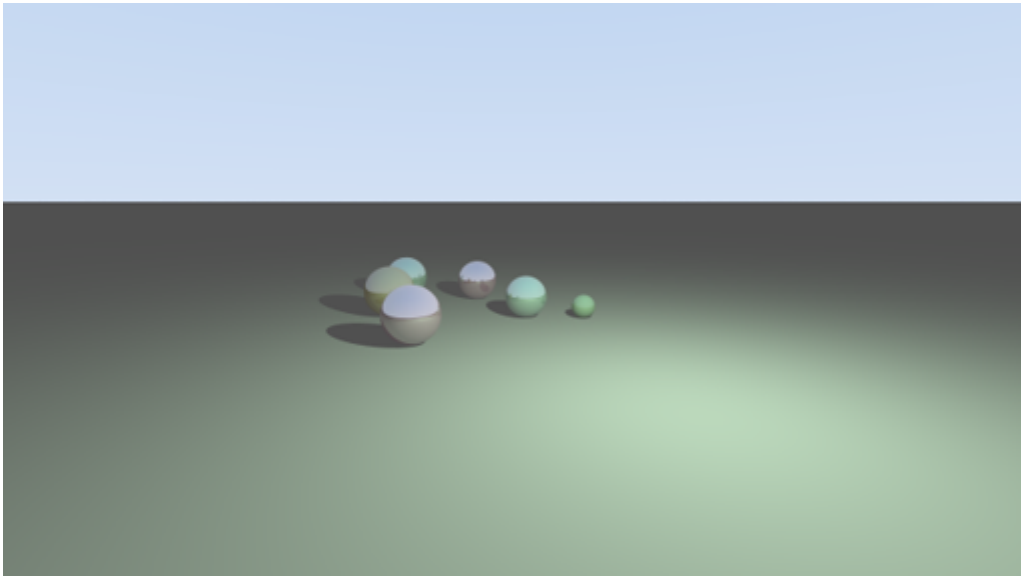
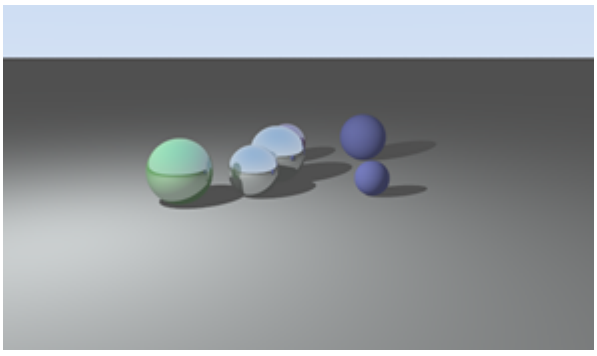
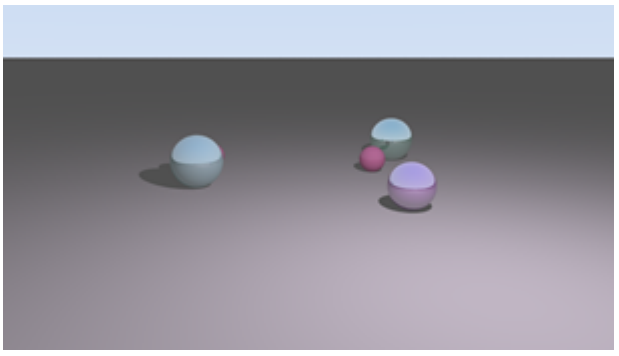


Figura 1. Escena de referencia (semilla 2073120322) generada por el ray tracer serial.

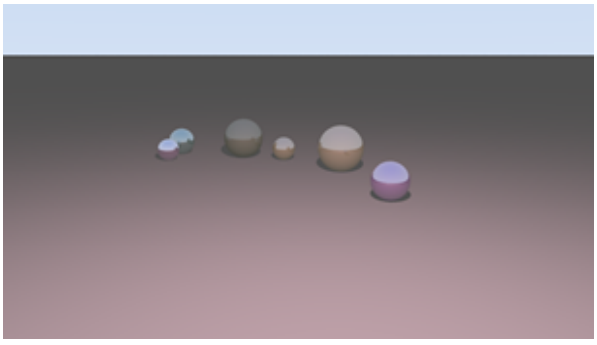
El programa también puede generar otras escenas cambiando la semilla. Algunos ejemplos:



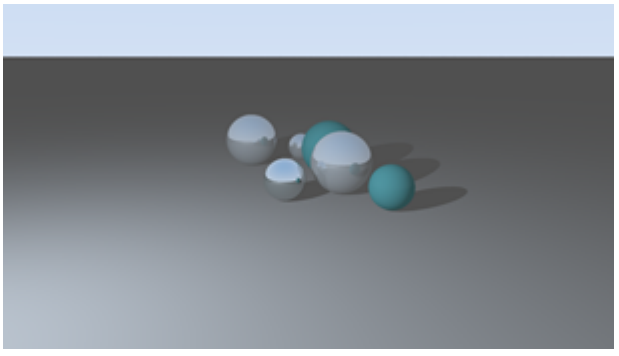
Semilla 1986051304 · luz a la izquierda



Semilla 1116340912 · luz a la derecha



Semilla 1810911669 · luz al centro, alta



Semilla 1300297273 · luz a la izquierda, baja

La consola muestra la semilla, la escena generada y el tiempo de cada ejecución:

```
====================================
       RAY TRACER SERIAL
====================================
Semilla: 2073120322
Resolucion: 1280 x 720
Muestras por pixel: 32
Profundidad maxima: 3

--- ESCENA ALEATORIA GENERADA ---
Esfera 1: posicion (0.49, -0.28, -5.12), radio 0.72, material 4
Esfera 2: posicion (-2.86, -0.15, -1.72), radio 0.85, material 2
Esfera 3: posicion (2.49, -0.59, -4.95), radio 0.41, material 0
Esfera 4: posicion (-4.48, -0.16, -8.88), radio 0.84, material 4
Esfera 5: posicion (-4.35, -0.12, -5.34), radio 0.88, material 3
Esfera 6: posicion (-1.45, -0.22, -8.52), radio 0.78, material 2
Luz: posicion (3.60, 4.72, 3.13)

Cantidad de esferas: 6

Renderizado terminado.
Tiempo serial: 6.9820 segundos
Imagen guardada: raytracer_seed_2073120322.ppm
```

Figura 3. Salida del programa con la semilla de referencia.



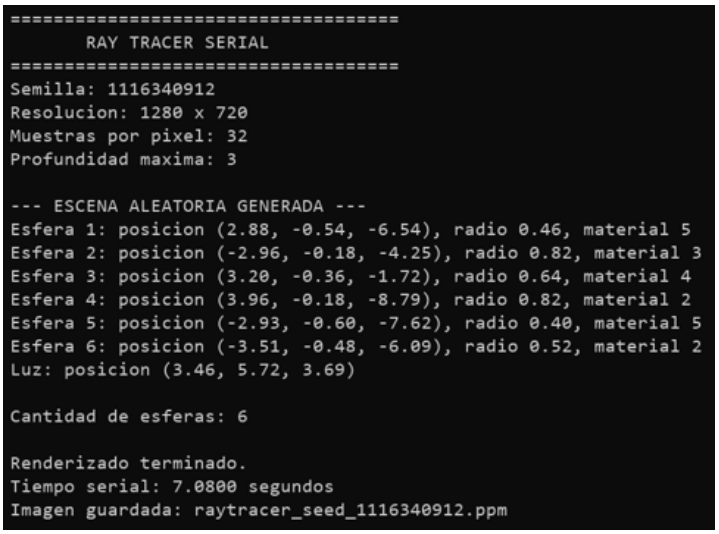


Figura 4. Salida con las semillas 1986051304 y 1116340912 (dos escenas de la Figura 2): cambian las esferas y la luz, pero el tiempo es parecido (7.03 s y 7.08 s).

Fase 2: Análisis PCAM

PCAM es un método de cuatro pasos para diseñar un programa paralelo: **dividir** el trabajo en tareas (Partición), ver qué datos deben **compartir** esas tareas (Comunicación), **juntar** las tareas pequeñas en bloques más grandes (Aglomeración) y **decidir** qué hilo hace cada bloque (Mapeo).

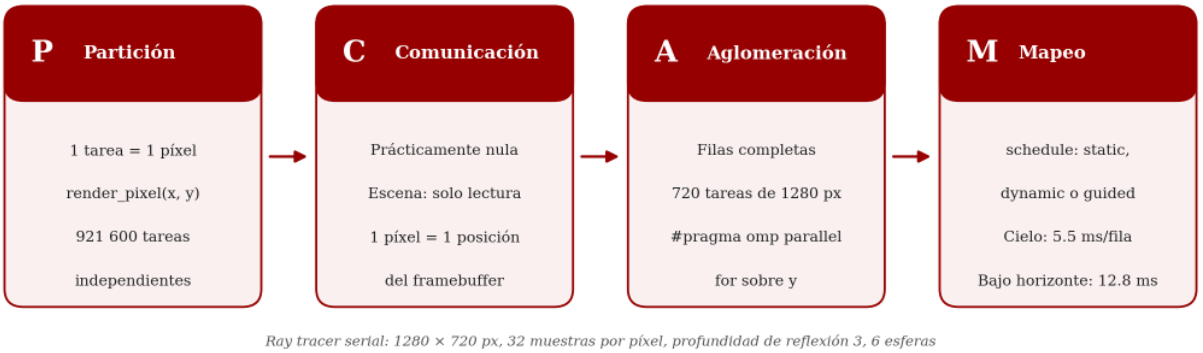


Figura 5. Resumen del análisis PCAM del ray tracer.

Etapa	Qué se decidió
Partición	Una tarea es calcular el color de un píxel. Hay 921 600 tareas independientes.
Comunicación	Prácticamente ninguna: la escena solo se lee y cada píxel escribe en su propia casilla.
Aglomeración	Agrupar por filas completas: 720 tareas de 1280 píxeles.
Mapeo	Repartir las filas entre los hilos con schedule. Las filas no cuestan lo mismo.

- **Partición:** Calcular el color de un píxel (función render_pixel(x, y)) es una tarea. La imagen tiene 921 600 píxeles y cada uno se puede calcular sin esperar a los demás. Cada tarea es muy pequeña: en promedio, un píxel tarda unos 7.6 microsegundos (7.008 s divididos entre 921 600 píxeles).
- **Comunicación:** La escena (esferas, materiales, suelo y luz) se crea antes de empezar a dibujar y después solo se lee, nunca se modifica. Cada píxel guarda su color en su propia casilla de la imagen, así que dos hilos nunca escriben en el mismo lugar.

Los números aleatorios que se usan para suavizar los bordes (antialiasing) dependen solo de la posición del píxel. Por eso la imagen sale exactamente igual sin importar cuántos hilos se usen, y la versión paralela se puede comparar byte a byte con la serial para comprobar que está bien.

- **Aglomeración:** Un píxel es tan poco trabajo que repartirlos de uno en uno costaría más de lo que se ahorra. Por eso se agrupan por **filas completas**: 720 filas de 1280 píxeles. Con 8 hilos, cada uno recibe unas 90 filas. En OpenMP esto se hace con #pragma omp parallel for sobre las filas.

Trabajar con filas completas también evita casi todo el *false sharing*, que ocurre cuando dos hilos escriben en datos que están en el mismo bloque de la memoria caché y se estorban entre sí. Con filas completas, eso solo podría pasar en el borde entre dos filas.

- **Mapeo:** En OpenMP esto se decide con schedule. Para elegir bien, se midió cuánto tarda cada fila (en una corrida aparte con la misma escena), y se encontró que **no todas las filas cuestan lo mismo**:

Zona de la imagen	Filas	Tiempo promedio por fila
Cielo	0 a 249	5.5 ms
Debajo del horizonte	250 a 719	12.8 ms

Las filas de abajo cuestan unas 2.3 veces más. Ahí cada rayo choca con el suelo o con una esfera, y el programa tiene que calcular la luz, lanzar un rayo de sombra y, en las esferas, calcular los reflejos. En el cielo el rayo no choca con nada y solo se pinta el color de fondo.

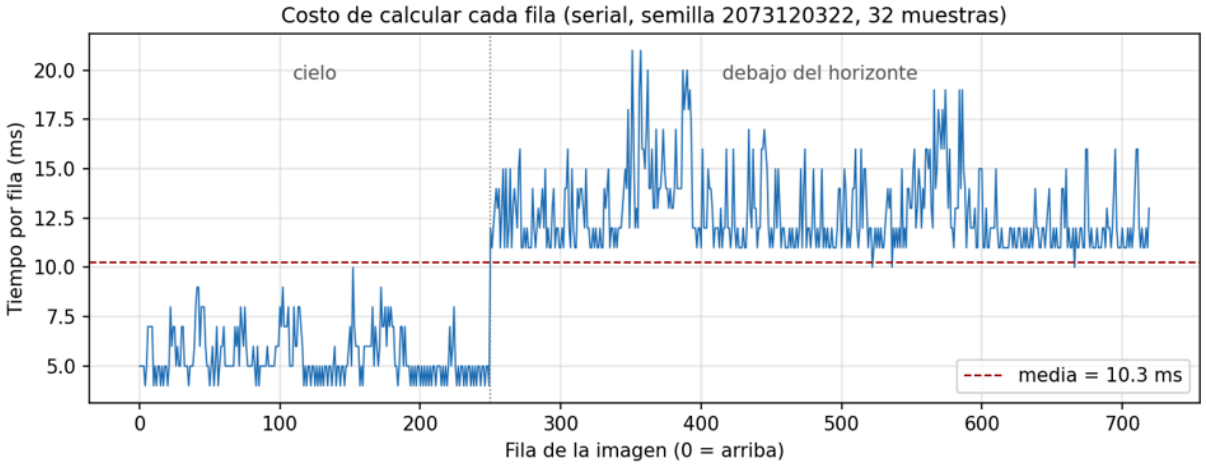


Figura 6. Tiempo de cada fila. Las filas del cielo (izquierda) son mucho más rápidas que las de debajo del horizonte.

Nota: el reloj mide con una precisión de cerca de 1 ms, por eso la gráfica se ve en escalones. Los promedios por zona sí son confiables.

Qué significa para el reparto entre hilos

Con schedule(static), OpenMP le da a cada hilo un bloque de filas seguidas antes de empezar. Los hilos que reciben filas del cielo terminan antes y se quedan esperando a los que reciben la parte de abajo. Con schedule(dynamic), cada hilo pide una fila nueva cuando termina la anterior, y así el trabajo se reparte mejor.

El **speedup** indica cuántas veces más rápido va el programa paralelo que el serial (tiempo serial ÷ tiempo paralelo). Con los tiempos por fila medidos se calculó el speedup **máximo posible** de cada forma de repartir, sin contar lo que cuesta manejar los hilos. Es una estimación, no una medición:

Hilos	static (bloques seguidos)	static,1 (filas alternadas)	dynamic,1
2	1.60	1.99	2.00
4	3.16	3.93	3.99
8	5.96	7.70	7.94

Hipótesis: static rendirá peor que dynamic o guided, sobre todo con 8 hilos (como máximo 5.96 frente a 7.94). Esto hay que confirmarlo midiendo el programa paralelo en las Fases 3 y 4.

Versión paralela con OpenMP

Todas las mediciones de esta fase se hicieron en este computador:

Dato	Valor en su equipo
Modelo de procesador	12th Gen Intel(R) Core(™) i5-1235U
Núcleos físicos (Core(s) per socket × Socket(s))	6
Hilos por núcleo	2
Núcleos lógicos (nproc)	12
Valor de _OPENMP	201511

Compilador: gcc [COMPLETAR VERSIÓN] (WSL, Ubuntu), con las opciones -O2 -fopenmp.
Sistema: Windows [COMPLETAR] con WSL.

Implementación

https://github.com/DSag31/Proyecto_Final_Programaci-n_Paralela/blob/main/RayTracer_Paralelo/raytracer_paralelo.c

La versión paralela usa **la misma escena, el mismo algoritmo y la misma función render_pixel** que la versión serial. Lo único que cambia es el bucle que recorre la imagen: en lugar de calcular un píxel después del otro, las filas (o los píxeles) se reparten entre varios hilos con OpenMP. La creación de la

escena y el guardado del archivo PPM siguen siendo seriales, y quedan fuera de la medición de tiempo.

1. Paralelización por filas y con collapse(2)

Siguiendo la aglomeración de la Fase 2, la primera versión reparte **filas completas** entre los hilos. Cada iteración del bucle es una fila de 1280 píxeles:

```
None
#pragma omp parallel
{
   #pragma omp for schedule(runtime) nowait
   for (int y = 0; y < HEIGHT; y++) {
      for (int x = 0; x < WIDTH; x++) {
         framebuffer[y*WIDTH+x] = render_pixel(x, y, camera);
      }
   }
}
```

Esto es equivalente a #pragma omp parallel for. Se separó en dos directivas (parallel y for) solo para que cada hilo pueda medir con omp_get_wtime() cuánto tardó y cuántos píxeles calculó, y así medir el **desbalance de carga**.

La segunda versión usa **collapse(2)**, que junta los dos bucles (y, x) en uno solo de 921 600 iteraciones. Así cada iteración es un píxel, y OpenMP puede repartir bloques de píxeles en lugar de filas completas:

```
None
#pragma omp for collapse(2) schedule(runtime) nowait
for (int y = 0; y < HEIGHT; y++)
   for (int x = 0; x < WIDTH; x++)
      framebuffer[y*WIDTH+x] = render_pixel(x, y, camera);
```

2. Comparación de schedule variando el chunk

El bucle usa schedule(runtime), y el tipo de reparto y el chunk se eligen al ejecutar el programa con omp_set_schedule(). Así se pueden probar todas las combinaciones **sin recompilar**:

```
None
./paralelo.exe [semilla] [hilos] [schedule] [chunk] [modo]
./paralelo.exe 2073120322 8 dynamic 1 filas
```

Se compararon estas configuraciones:

Configuración	Qué hace
static (bloques) · filas	Cada hilo recibe un bloque de filas seguidas antes de empezar.
static,1 · filas	Las filas se reparten alternadas: fila 0 al hilo 0, fila 1 al hilo 1, etc.
dynamic,1 · filas	Cada hilo pide una fila nueva cuando termina la anterior.
dynamic,4 · filas	Igual, pero pide 4 filas a la vez.
guided,1 · filas	Bloques grandes al principio que se van achicando hasta 1 fila.
static (bloques) · collapse	Bloques de píxeles seguidos, repartidos antes de empezar.
dynamic,64 · collapse	Cada hilo pide 64 píxeles a la vez.
guided,16 · collapse	Bloques decrecientes de píxeles, mínimo 16.

3. Generador aleatorio sin estado compartido

Como se explicó en la Fase 1, el antialiasing (32 muestras por píxel) no usa rand() ni ninguna variable global. Cada píxel crea su propio generador a partir de sus coordenadas:

```
None
uint32_t state = hash32((uint32_t)(x+1)*73856093u ^ (uint32_t)(y+1)*19349663u);
```

Por eso los números aleatorios de un píxel dependen solo de su posición (x, y), y no de qué hilo lo calcula ni en qué orden. No hay condiciones de carrera, no hace falta critical, atomic ni lock, y el resultado es **determinista**: la imagen sale igual con 1, 2, 8 o 12 hilos.

4. Prueba de correctitud: imagen idéntica byte a byte

Para comprobar que la versión paralela está bien, se generó la imagen con la versión serial y con cada configuración paralela usando la misma semilla (2073120322), y se compararon los dos archivos byte a byte con el comando cmp:

```
None
./serial.exe 2073120322
./paralelo.exe 2073120322 8 dynamic 1 filas
cmp raytracer_seed_2073120322.ppm raytracer_paralelo_seed_2073120322.ppm && echo "IDENTICAS"
```

Todas las configuraciones produjeron una imagen idéntica a la serial. El script de mediciones (benchmark_fase3.sh) hace esta comprobación antes de medir y se detiene si alguna imagen no coincide. Además, el programa imprime una suma de verificación de la imagen, que sirve para comparar rápido sin abrir los archivos.

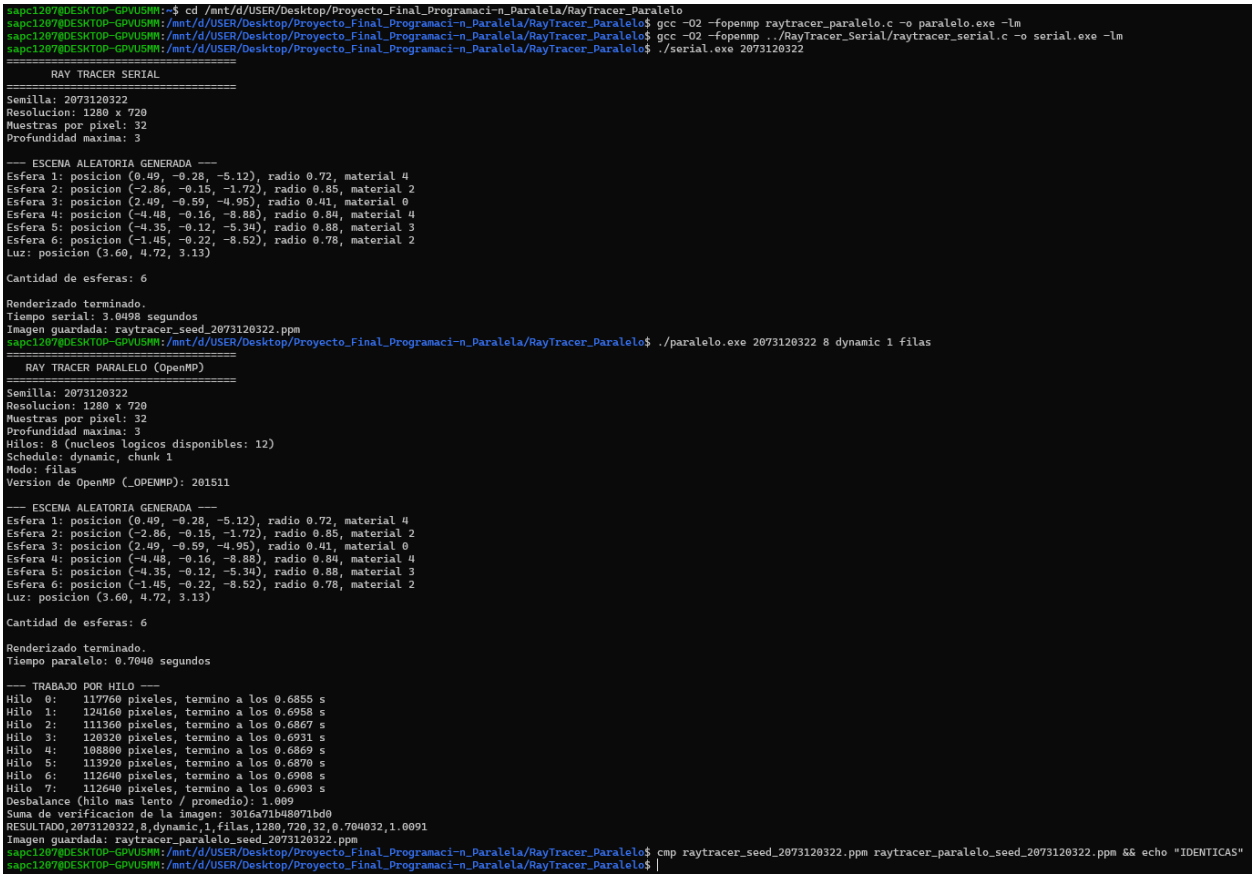


Figura 7. Salida de la versión paralela: tiempo, píxeles por hilo, desbalance y verificación contra la versión serial.

5. False sharing

El *false sharing* ocurre cuando dos hilos escriben en datos distintos que están en la misma línea de la memoria caché (64 bytes). Aunque no se pisan los datos, el procesador tiene que sincronizar esa línea entre núcleos y el programa se vuelve más lento.

Cada píxel del framebuffer ocupa 24 bytes (un Vec3 con tres double), así que en una línea de caché caben unos 2,7 píxeles:

- **Por filas:** cada hilo escribe filas completas de 1280 × 24 = 30 720 bytes (unas 480 líneas de caché). Solo pueden compartir línea los extremos de dos filas vecinas, así que el false sharing es despreciable.
- **Con collapse(2) y chunk 1:** píxeles vecinos van a hilos distintos, y varios hilos escriben en la misma línea de caché. **Este es el caso con false sharing.** Por eso con collapse se usaron chunks de 16, 64 o bloques grandes.

- **Contadores por hilo:** cada hilo cuenta sus píxeles en una variable local y la guarda en un arreglo compartido una sola vez al final, en lugar de actualizar el arreglo en cada iteración. Así se evita que los contadores generen false sharing.

En este programa el efecto es pequeño incluso en el peor caso: cada píxel tarda microsegundos en calcularse (32 rayos con hasta 3 rebotes) y se escribe una sola vez, así que las escrituras son muy pocas comparadas con el cálculo.

6. Primera comparación de los schedule

Cada configuración se ejecutó 5 veces con 1, 2, 4 y 8 hilos y se tomó la mediana. El tiempo serial de referencia en este equipo fue de **2.974 s**. Resultados con 8 hilos:

Configuración	Tiempo (s)	Speedup	Eficiencia	Desbalance
static (bloques) · filas	1.112	2.68	0.33	**1.30**
static,1 · filas	**0.999**	**2.98**	**0.37**	1.02
dynamic,1 · filas	1.125	2.65	0.33	1.01
dynamic,4 · filas	1.088	2.73	0.34	1.03
guided,1 · filas	1.069	2.78	0.35	1.01
static (bloques) · collapse	1.364	2.18	0.27	**1.28**
dynamic,64 · collapse	1.073	2.77	0.35	1.00
guided,16 · collapse	1.083	2.75	0.34	1.00

El desbalance es el tiempo del hilo más lento dividido entre el promedio de todos los hilos. 1.00 significa un reparto perfecto.

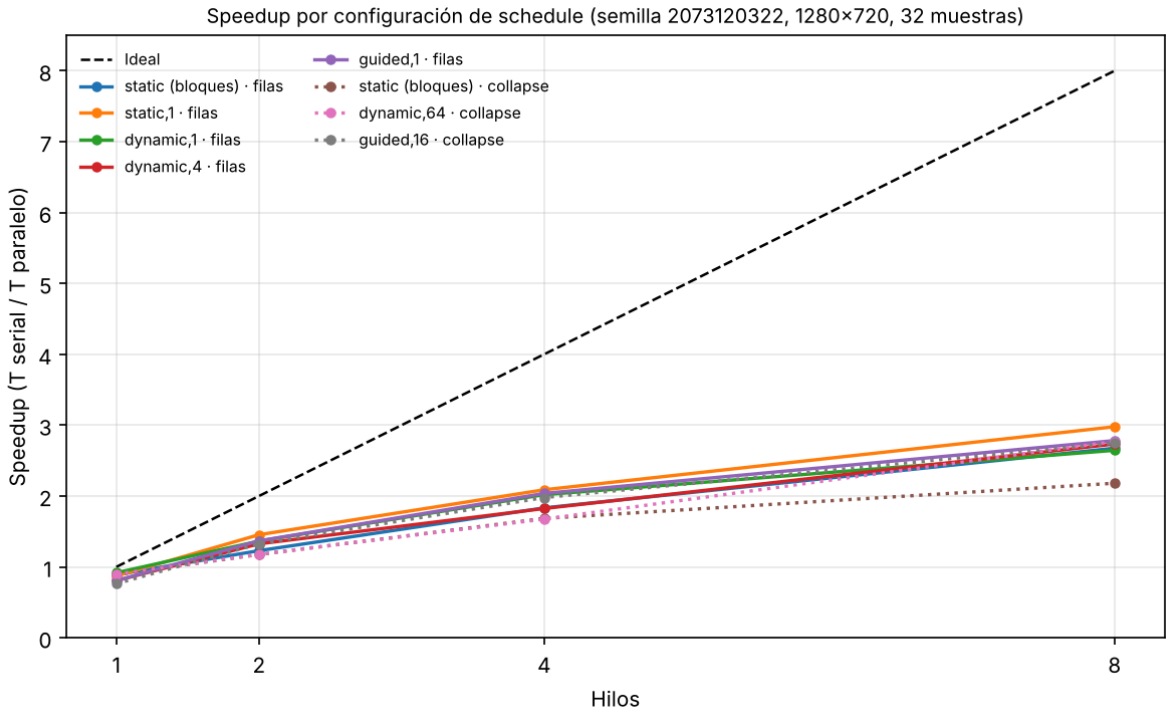


Figura 8. Speedup de cada configuración según el número de hilos, comparado con el speedup ideal.

Qué muestran los resultados:

- **La hipótesis de la Fase 2 sobre el desbalance se confirma.** Con static en bloques, un hilo tarda un 30 % más que el promedio (desbalance 1.30), porque recibe las filas de debajo del horizonte, que cuestan 2.3 veces más que las del cielo. Con dynamic, guided o static,1 el desbalance baja a casi 1.00.
- **La mejor configuración fue static,1 por filas** (2.98× con 8 hilos). Repartir las filas alternadas mezcla las filas caras con las baratas, y logra el mismo balance que dynamic sin el costo de que los hilos pidan trabajo una y otra vez.
- **La peor fue static en bloques con collapse** (2.18×), por la misma razón del desbalance.
- **El speedup real (~3× con 8 hilos) está muy por debajo del estimado en la Fase 2 (7.94×).** Como el ray tracer casi no tiene parte serial, la diferencia no viene del algoritmo sino del hardware: con varios hilos activos el procesador baja su frecuencia (turbo), y los hilos lógicos que comparten un mismo núcleo físico (hyper-threading) aportan poco en un cálculo con tantas operaciones en double. Esto se analiza en la Fase 4.
- **Con 1 hilo, la versión paralela es un poco más lenta que la serial** (0.76× a 0.92×). Esa diferencia es el costo de OpenMP: crear el equipo de hilos, el reparto en tiempo de ejecución y el registro de tiempos por hilo.

Experimentos y medición

Todas las mediciones se hicieron en un mismo equipo (Tabla 1).

Tabla 1. Configuración del equipo de pruebas.

Elemento	Valor
Memoria RAM	24GB
Sistema operativo	Windows 11
Modo de energía	Alto rendimiento
Hilos evaluados	1, 2, 4 y 8
Semilla del render	2073120322

Todas las mediciones de esta fase se hicieron en este computador:

Dato	Valor en su equipo
Modelo de procesador	13th Gen Intel(R) Core(TM) i5-13420H
Núcleos físicos (Core(s) per socket × Socket(s))	8 × 1 = 8 (4 de rendimiento y 4 de eficiencia)
Hilos por núcleo	2 en los 4 núcleos de rendimiento (hyper-threading) y 1 en los 4 de eficiencia
Núcleos lógicos (nproc)	12
Valor de _OPENMP	OpenMP 4.5

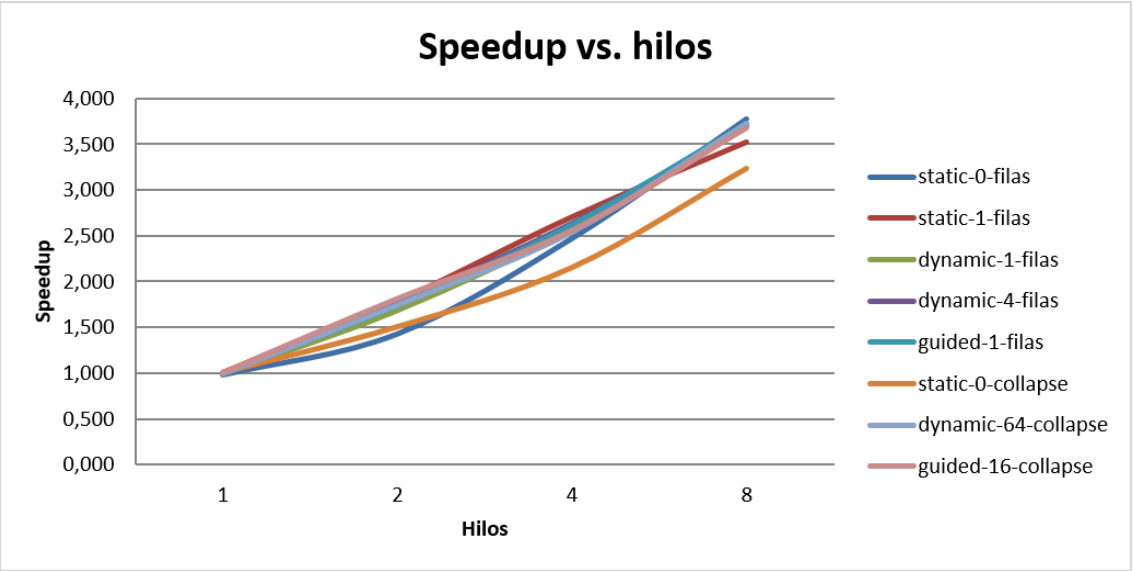
Análisis

Balance de carga. El costo de renderizar no es uniforme: las filas con esferas, reflejos y sombras cuestan más que las de cielo. Los repartos estáticos por bloques concentran esas filas en pocos hilos y quedan con un desbalance de 1.2 a 1.3. Con 2 hilos su speedup fue de 1.43 contra 1.68–1.81 de las configuraciones balanceadas, y con 4 hilos fue de 2.47 contra 2.54–2.71. static-0-collapse fue la peor configuración en todos los tamaños (3.24× con 8 hilos). Las demás quedaron con desbalance de 1.00 a 1.05.

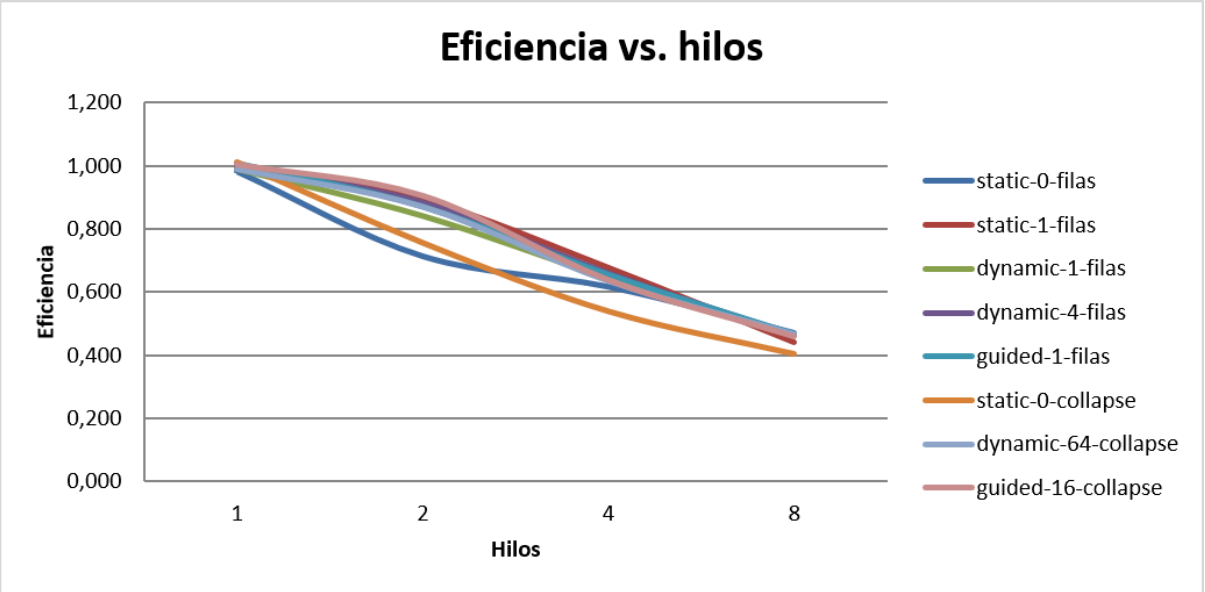
Comparación entre esquemas balanceados. Con 8 hilos las seis configuraciones balanceadas dieron entre 0.801 y 0.846 s, una diferencia de 5.6 % (Tabla 3). La variación entre corridas llegó hasta 8.9 % de CV, así que con 5 corridas no se puede nombrar un ganador. Que static-0-filas haya salido primero a 8 hilos (0.790 s) pese a su desbalance no debe leerse como superioridad, porque su ventaja está dentro del ruido.

configuracion	Mediana (s)	Speedup	Eficiencia	Desbalance prom.	CV (%)	Min (s)	Máx (s)
static-0-filas	0,7900	3,7759	0,4720	1,2724	3,27	0,7510	0,8180
static-1-filas	0,8460	3,5260	0,4408	1,0463	8,92	0,7020	0,8980
dynamic-1-filas	0,8050	3,7056	0,4632	1,0058	3,43	0,7810	0,8500
dynamic-4-filas	0,8080	3,6918	0,4615	1,0220	0,94	0,7930	0,8110
guided-1-filas	0,8010	3,7241	0,4655	1,0050	5,96	0,7800	0,9020
static-0-collapse	0,9220	3,2354	0,4044	1,2981	5,27	0,8720	0,9900
dynamic-64-collapse	0,8010	3,7241	0,4655	1,0007	2,43	0,7650	0,8170
guided-16-collapse	0,8120	3,6736	0,4592	1,0004	3,40	0,7670	0,8210

Escalabilidad.



Con 8 hilos el speedup llegó a ~3.7× frente al serial de 2.983 s (Figura 1). La eficiencia cae de 0.84–0.91 con 2 hilos a 0.63–0.68 con 4 y a 0.44–0.47 con 8 (Figura 2). Pasar de 4 a 8 hilos solo mejoró el speedup entre 1.3× y 1.5×, cuando duplicar los recursos daría 2×.



Esto se relaciona con la arquitectura híbrida del procesador. Los 8 hilos no son equivalentes entre sí: según cómo los reparta el sistema operativo, pueden compartir un núcleo de rendimiento (dos hilos con los mismos recursos) o caer en núcleos de eficiencia, que son más lentos. En ambos casos cada hilo adicional aporta menos que el anterior, y por eso la curva se aplana. La eficiencia a 8 hilos (~0.46) debe leerse con esa salvedad: se calcula sobre 8 hilos que no rinden lo mismo, y no sobre 8 núcleos idénticos.

La eficiencia ya baja de ~0.88 a ~0.65 entre 2 y 4 hilos, antes de que los hilos tengan que compartir núcleo. En un portátil de la serie H, la frecuencia suele bajar cuando hay más núcleos activos, por los límites de potencia y temperatura, y esa es una causa probable. No se midió la frecuencia ni se fijó la afinidad de hilos, así que no puede confirmarse.

Fracción serial. Karp-Flatt se estabiliza en ~0.16–0.18 con 4 y 8 hilos, lo que sugiere un límite fijo del orden de 16–18 % y un techo de Amdahl de ~5.5 a 6×. Como el tiempo se mide solo sobre el cómputo, esa fracción no corresponde a la inicialización ni a la escritura a disco. Además, en un procesador híbrido mezcla la parte secuencial real con el efecto de que los núcleos no sean iguales, por lo que debe leerse como una fracción serial efectiva.

Costo de OpenMP. Con 1 hilo todas las configuraciones quedaron entre 0.98× y 1.01× del serial, así que el sobrecosto de la paralelización es despreciable.

Conclusiones generales

- **La paralelización redujo el tiempo de forma significativa.** Con 8 hilos el render pasó de 2.983 s a ~0.8 s (~3.7×). Con 1 hilo todas las versiones quedaron entre 0.98× y 1.01× del serial, así que el sobrecosto de OpenMP es despreciable y la mejora se debe al reparto del trabajo.
- **El balance de carga es el factor que más afecta el rendimiento.** El costo por fila es irregular, y el reparto estático por bloques deja unos hilos con más trabajo que otros (desbalance de 1.2 a 1.3), sobre todo con 2 y 4 hilos. Combinado con collapse es la peor configuración. En un problema con costo irregular conviene un reparto cíclico o por demanda.
- **Entre los esquemas balanceados no hay un ganador claro, pero se recomienda guided-16-collapse.** Con 8 hilos la diferencia entre ellos (5.6 %) es comparable al ruido de medición. Esta configuración tuvo el mejor speedup con 2 hilos (1.81), desbalance prácticamente nulo y la menor variabilidad (CV de 1.6 a 3.5 %). static-1-filas es una alternativa válida, más simple y con menos sobrecosto de planificación, pero fue más variable con 8 hilos.
- **La escalabilidad se aplana por el procesador y por una fracción serial efectiva. El speedup llega a ~3.7× con 8 hilos.** El procesador es híbrido (8 núcleos, 12 hilos), y los hilos adicionales comparten núcleo o caen en núcleos más lentos, así que cada uno aporta menos. Además, Karp-Flatt se estabiliza en ~0.16–0.18, lo que limita el speedup máximo a ~5.5–6× según Amdahl. La caída de eficiencia ya aparece entre 2 y 4 hilos, lo que apunta también a límites de frecuencia o potencia.
- **El límite no lo pone el scheduler, y quedan variables por explorar.** Las configuraciones balanceadas terminan con la misma fracción serial efectiva, así que cambiar de esquema no mueve el techo. Falta medir la inicialización y la escritura por separado, registrar la frecuencia, fijar la afinidad de hilos y probar con 12 hilos para usar todos los procesadores lógicos. También falta variar la resolución, las muestras por píxel y la complejidad de la escena, y usar la partición cpu del clúster para extender la curva.